



Prompt Claude Code — Outil de veille & idées de vidéos quotidiennes

À coller dans **Claude Code** pour construire un outil qui, **chaque matin**, analyse l'actu tech + ce qui buzze sur TikTok/YouTube/Reels et te **propose des idées de vidéos directement dans Notion**, calibrées sur ta ligne éditoriale.

Deux blocs : **(A)** le prompt de build (crée l'outil une fois) · **(B)** le prompt du run quotidien (ce que l'outil exécute chaque matin). Copie le bloc A dans Claude Code pour lancer la construction.

A. PROMPT DE BUILD (à coller dans Claude Code)

Tu vas me construire un outil CLI en Python appelé "veille-video" qui tourne chaque matin analyse l'actu tech et ce qui buzze sur les réseaux, et propose des idées de vidéos directement dans ma base Notion "Scripts", calibrées sur ma ligne éditoriale.

Contexte créateur (ma DA – à respecter à la lettre)

- Chaîne : étudiant ingénieur (INSA Lyon) qui documente sa montée en puissance en tech/IA build in public, + décryptage d'entrepreneurs. Objectif : ~50 000 abonnés.
- Équilibre 70% grand public (actu IA, grandes boîtes, bourse/IP0, innovations qui font parler – façon Léo Duff) / 30% niche perso (étudiant ingé, code, build in public).
- Critère de tri unique d'une idée : "est-ce que ça peut faire BEAUCOUP de vues ?"
- Styles possibles : Léo Duff, Léo Duff+Vinceeh (art×ingé), Moubeche (build in public), Micod (enquête, je montre), Ego (essai réflexif qui fait réfléchir), Vlog, Finary.
- Règles : titre = vraie question honnête (zéro putaclic) ; nuance obligatoire ; edge sans cible ; la vidéo doit faire réfléchir. INTERDIT ABSOLU : achat-revente luxe/seconde main (resell, sneakers, Vinted, Superbuy...) – ne jamais proposer ce thème.
- Le guide complet d'écriture est dans docs/GUIDE-ECRITURE-VIDEOS.md : lis-le et sers-t'en comme référence de calibrage pour chaque idée.

Ce que l'outil doit faire, chaque matin

1. VEILLE ACTU TECH (dernières 24-36 h) :

- Récupère les gros titres via des sources fiables et gratuites : flux RSS de The Verge, Ars Technica, TechCrunch, MIT Tech Review, Hacker News (front page API Firebase : <https://hacker-news.firebaseio.com/v0/topstories.json>), et l'API publique Reddit (r/technology, r/artificial, r/singularity : <https://www.reddit.com/r/<sub>/top.json>) Utilise WebSearch/WebFetch pour compléter et vérifier.
- Pour la bourse/IP0/grandes boîtes : titres éco tech du jour (WebSearch).

2. VEILLE "CE QUI BUZZE" (formats + angles qui marchent MAINTENANT) :

- TikTok : identifie les vidéos/sujets tech & business qui performant. Deux voies, dans cet ordre de préférence :
 - (a) si une clé TikTok API (Research/Display) est présente dans .env, utilise-la ;
 - (b) sinon, fallback robuste : WebFetch sur les pages hashtag/recherche TikTok (ex : <https://www.tiktok.com/tag/ia> , /tag/intelligenceartificielle , /tag/tech /tag/startup) et WebSearch "TikTok tech viral <date>" pour extraire les angles, hooks et formats qui tournent. Si le scraping échoue (anti-bot), ne bloque pas, logue l'échec et continue avec les autres sources.
- YouTube : tendances tech FR via l'API YouTube Data v3 (search + videos, order=viewCount, publishedAfter=24h, regionCode=FR) si YOUTUBE_API_KEY est dans .env ; sinon WebSearch. Repère surtout les VIDÉOS RÉCENTES à forte vue de mes inspirations : Léo Duff, Micod, Ego, Vinceeh, Underscore, Marc Lou, Pieter Levels.
- Reels/Shorts : angles courts qui buzzent (via WebSearch si pas d'API).

3. GÉNÈRE 5 IDÉES DE VIDÉOS calibrées :

- Répartition : ~3 grand public (moteur) + ~1 actu chaude 🔥 + ~1 niche perso 🎯.
- Chaque idée doit être NEUVE : avant de proposer, interroge ma base Notion "Scripts" le "Backlog d'idées" (via le MCP Notion) et écarte tout doublon de thème/angle déjà.
- Pour chaque idée, produis :
 - Titre = une vraie question honnête (R9), avec parenthèse-enjeu si pertinent.
 - Style dominant recommandé (parmi la liste ci-dessus) + pourquoi.
 - Pilier(s) : Étudiant-INSA / IA-code / Décryptage / Mindset / Build in public / A.
 - Plateforme cible : YouTube long / YT Short / TikTok / Reel.
 - Le HOOK (3 variantes, dont un paradoxe/contraste).
 - Le message (la phrase qui reste) en une ligne.

- L'angle de nuance obligatoire (R8).
 - "Pourquoi ça peut faire des vues MAINTENANT" : le signal d'actu/buzz qui la justifie (lien source + métrique si dispo : X vues sur tel TikTok, front page HN, etc.).
 - 2-3 sources vérifiées (URL).
 - Un score viralité /10 (basé sur : volume du sujet, tension du hook, fraîcheur actuelle, potentiel de découpe en Shorts) + une phrase de justification.
- Trie les idées par score décroissant.

4. ÉCRITURE DANS NOTION :

- Utilise le serveur MCP Notion. Crée les pages dans la data source "Scripts" (collection://37e2c750-dble-810b-b9e7-000b197bc796) avec Statut="Idée".
- Renseigne les propriétés : Titre, Style, Pilier (multi), Plateforme, Statut="Idée", Date création = aujourd'hui.
- Le CONTENU de chaque page suit le template de docs/GUIDE-ECRITURE-VIDEOS.md (cold open, 3 hooks, message, ce qu'on remet en question, structure, nuance, chute, shorts, sources, + le bloc "Pourquoi ça buzze maintenant" avec les liens).
- Ajoute en tête un callout récap du jour : date, top idée, score, thèmes couverts.
- Ne crée RIEN si une idée est un doublon manifeste ; logue-le à la place.

5. LIVRABLE LOCAL :

- Écris aussi un fichier daté dans ./out/veille-YYYY-MM-DD.md avec les 5 idées complètes (même contenu que Notion) pour archive et lecture rapide.
- Affiche en fin de run un résumé console : nb d'idées créées, top idée + score, liens

Contraintes techniques

- Python 3.11+, dépendances minimales (requests, feedparser, python-dotenv). Pas de secrets.
- Toutes les clés dans un .env (TIKTOK_*, YOUTUBE_API_KEY optionnelles). Fournis un .env.
- Robustesse : chaque source dans un try/except isolé ; une source qui tombe ne casse jamais. Timeout réseau court + retries avec backoff. Cache 24h des flux pour éviter les doublons.
- Idempotence : si relancé le même jour, ne recrée pas les mêmes idées (garde un state local dans ./out/state.json avec les titres déjà proposés).
- Fournis un README.md : install, config .env, lancement manuel, et la commande cron.
- Écris quelques tests (parsing des flux, dédup, calibrage) et fais-les passer.

Planification (chaque matin)

- Fournis une entrée cron pour 7h00 heure de Paris, ex :
`0 7 \* \* \* cd /chemin/veille-video && /usr/bin/python3 run.py >> out/cron.log 2>&1`
- Explique comment installer le cron (crontab -e) et comment tester le run à la main d'abord.

Commence par me proposer l'arborescence du projet et le plan, puis code, teste, et fais un premier run "dry-run" (sans écrire dans Notion) que je validerai avant d'activer l'écriture.

B. Le run quotidien, en une phrase (résumé mental de ce que fait l'outil)

Chaque matin à 7h : il lit l'actu tech des dernières 24 h + repère ce qui buzze sur TikTok/YouTube/Reels, croise avec ma DA (70/30, styles, règles, sujet interdit), écarte les doublons de mon backlog Notion, et dépose **5 idées de vidéos prêtes à écrire** (titre-question, 3 hooks, message, nuance, structure, shorts, sources, score viralité) dans ma base **Scripts** + un fichier d'archive local daté.

Notes de mise en place

- **MCP Notion** : l'outil doit tourner dans un environnement où le serveur MCP Notion est connecté (comme cette session), ou via l'API Notion officielle avec un token d'intégration ajouté à la data source "Scripts". Le plus simple : lancer le run quotidien **via Claude Code** (qui a déjà le MCP Notion), avec un cron qui invoque Claude Code sur le prompt du bloc B.
- **TikTok** : l'accès programmatique officiel (Research API) demande une validation ; le fallback WebFetch/WebSearch fonctionne sans clé mais est moins précis sur les métriques exactes. C'est volontairement dégradé-résistant : mieux vaut un angle repéré sans compteur de vues exact qu'un run qui plante.
- **Sécurité/robustesse** : aucune source n'est bloquante ; le run produit toujours au moins quelques idées à partir de l'actu tech même si les API sociales échouent.

ID data source Notion "Scripts" : `collection://37e2c750-db1e-810b-b9e7-000b197bc796` . Guide de calibrage : `docs/GUIDE-ECRITURE-VIDEOS.md` .